Twenty reusable control proofs for K215

Guarded tape frames, flexible return states, and checked source attachments

Research and integration note for Vladimir Reshetnikov's ProveIt project

23 October 2026

Abstract

We propose a concrete next step in simplifying the correctness proofs of the accepted 215-state consistency machine: prove a control instruction on its smallest relevant suffix, transport that proof across an untouched left tape frame, and finish through a separately checked return word. An earlier repository proposal identified 35 local traces but left the general frame proof uncompiled. We complete that mechanism and improve its decomposition: allowing return state 1 as well as state 2 reduces the library to 20 traces, with 239 symbolic operations, covering 1,423 of 1,425 control occurrences. The remaining control occurrences are left on the existing literal proof route.

The accompanying Lean component proves the guarded frame rule, its positive-run semantics, the generalized Home composition, source-path framing, exact tape endpoints, and finite caller attachments against the maintained definitions. The concrete register-engine and Home premises are discharged. A separate Coq 8.18 compatibility component checks the corresponding symbolic rule and the same twenty traces against verbatim-extracted Rocq definitions; it does not replace a Rocq 9.1 endpoint build. The archive includes source, generators, independent finite audits, input hashes, raw compiler output, and reproduction instructions.

The proposed expensive execution-obligation count is 4,773 rather than 6,176, while every source-coverage outcome remains required. This is a structural reduction, not a measured end-to-end speedup or a completed replacement of the published physical-machine endpoint. The report also corrects the working baseline: the pinned source tree already contains computed decoder trees, direct C222 compilation, and the direct full-seed construction. Those earlier improvements are not claimed again.

Contents

1 Recommendation and precise scope **3**

2 Repository baseline and theorem being preserved **3**
2.1 Pinned input . . . 3
2.2 The current tree is newer than the phase-fifteen report . . . 4
2.3 Logical target . . . 4

3 Why the runtime control layer is the right target **4**

4 Symbolic configurations and the guarded frame rule **5**
4.1 The maintained representation . . . 5
4.2 Why left normalization is forbidden here . . . 6

5 Finite replay and strictly positive execution **6**

6 The improvement: a flexible return-state contract **7**
6.1 Read-only return words . . . 7
6.2 Generalized Home completion . . . 7
6.3 A three-step example serving 181 sites . . . 8

7 Source-path framing and exact gap arithmetic **8**
7.1 Uniform gap parameter . . . 9

8 The complete twenty-template library **10**

9 Exact caller attachment and whole-tape contracts **10**

10 The Rocq-compatible component **11**

11 Fresh validation and trust boundaries **12**

12 Integration plan **13**
12.1 First preserve a small stable interface . . . 13
12.2 Attach to semantic residual coverage . . . 13
12.3 Maintain one generator and one finite format . . . 13
12.4 Finish with the actual endpoint clients . . . 14

13 What the reduction measures **14**

14 Further research questions **14**

15 Artifact guide and reproduction **15**

1 Recommendation and precise scope

The recommended next integration is *framed semantic control contracts*. Keep the existing machine, source program, decoder proof, register engine, and ZFC calibration. Replace repeated control-tail executions by applications of a small library whose entry state, source suffix, output suffix, return state, and whole-tape behavior are explicit. Supply the existing literal proofs for the remaining instruction families until a separate simplification of those families is justified.

The delivered contribution is more than a proposed tactic. Its reusable Lean argument and finite local traces are freshly kernel checked, and its caller interface retains the exact source operation and full decoded path. The mechanism is deliberately narrow: it proves a mathematical relation between complete configurations. It does not attempt to infer correctness merely from equal register values, matching state names, or successful bounded experiments.

Measure	Earlier frame proposal	This component
Distinct local traces	35	20
Symbolic operations in the trace library	444	239
Control occurrences represented	1,422	1,423
Continue occurrences represented	1,326	1,327
Break occurrences represented	96	96
Literal control fallbacks	3	2
Candidate expensive execution obligations	4,789	4,773

The first column is credited to the repository's `round13-left-frame` proposal [3]. The new generalization is to treat the terminal state of the local body as part of a return-word contract. In particular, state 1 can perform part of the return scan; requiring every local body to reach state 2 had unnecessarily retained bits of the prefix inside several templates.

2 Repository baseline and theorem being preserved

2.1 Pinned input

All inspected inputs are pinned to ProveIt commit

`cbf94cb552a3b49509e0eee80e7455d4eb5974c6`.

The accepted dense transition table has SHA-256

`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090`.

The accepted native sparse table has SHA-256

`9683d6ed9628e6ce407c315b08f5da3d063da94e92706f275ca0f33e8272e4b2`.

The formal sparse labels map native states 301, 302, and 254 to 102, 120, and 230 respectively. The audit compares all 430 actions after this renaming. The distinction matters: native state 302 occurs in local control traces, so simply copying native numeric labels into a Lean certificate would certify the wrong statement.

The machine remains a two-symbol, one-tape, two-way-infinite machine with 215 working states, blank input, and a separate halting state. Initialization is included in the working-state count. The delivered work makes no change to this table.

2.2 The current tree is newer than the phase-fifteen report

The verification README records the historical complete Lean and Rocq endpoint checks. The phase-fifteen checkpoint documents 176 Lean project modules and 2,052,142 selected source bytes [1, 2]. A fresh static import census of the actual pinned `KEndpoint215` source instead gives 174 project modules, 1,752,650 source bytes, and 37,648 physical lines. Its only external toolchain import is `Init`; all project imports were resolved.

This difference is explained in part by subsequent adopted changes:

1. Commit `775267c38a` computes both decoder trees from their programs. In particular, `KSource215Program.lean` is already 6,867 bytes, and `t215 := ptree p215` is already maintained. The two K215 literal-tree removals save 122,874 and 122,911 bytes in their respective files [4].

2. Commit `fa92b88f8e` compiles C222 directly in the 215 and 217 enumerators through the existing dominated-implication theorem. The intermediate CDom compiler route is already removed from those endpoint import closures [5].

3. Commit `ff928ef0ec` supplies the full seed directly on the original model. The preliminary equality-free/indiscernibility route is already replaced in the maintained lower bridge [6].

These commit messages describe focused successful checks and explicitly leave broader physical-endpoint rebuilds to subsequent work. Consequently, three facts must be kept separate: a declaration is present in current sources; a focused component has been checked; a complete endpoint receipt binds a particular historical source closure. The present census measures current source text. It is not a newly completed 174-module endpoint build. The package's fresh compiler evidence covers the new component and its actual prerequisites.

This baseline check prevents a misleading proposal to "remove" trees or intermediate calculi that are already gone, and prevents counting historical reductions as new savings.

2.3 Logical target

The endpoint under discussion is the equivalence

$$\mathrm{Halt}(K215, \mathrm{blank}) \quad \Longleftrightarrow \quad \text{there is a finite ZFC derivation of falsity.}$$

Negating both sides gives nonhalting if and only if syntactic consistency. Neither direction assumes consistency as a premise. The repository names include Lean's `KEndpoint215.blank_halts_iff_zfc_refutation` and Rocq's `tm215_halts_iff_ZFC_inconsistent` [1].

The existing verification documentation carefully distinguishes these external proof-assistant theorems from a derivation explicitly encoded in ZFC's own proof syntax. This report preserves that distinction. It supplies no new internal ZFC or PA derivation, no unconditional nonhalting theorem, and no change to the consistency calibration.

3 Why the runtime control layer is the right target

The current simulator carries an exact program counter on the tape. After decoding a source word, the primitive control action starts at a routed state and acts near the end of that word. Much of the program-counter prefix is unchanged. Nevertheless, literal site proofs may execute a substantially similar suffix and return scan separately at many source occurrences.

The complete source has 4,751 expanded leaves and 6,176 outcome checks. The latter count exceeds the leaf count because decrement has two outcomes. The instruction census is

$$1,900 \text{ increments} + 2,850 \text{ decrement outcomes} + 1,425 \text{ controls} + 1 \text{ halt}.$$

The finite audit independently re-expands the source DAG and matches the accepted site data. It also follows all 4,751 complete decoder routes through the current 234 roles. Thus the grouping is attached to actual source occurrences, rather than inferred from a physical entry state alone.

The appropriate proof boundary is a semantic contract. A successful whole-tape execution theorem need not imply acceptance by the old 4,000-fuel Boolean checker `checkEdge`. The existing semantic-coverage and exact residual proposals already make this distinction [7, 8]. Our contracts are intended to feed that semantic interface. Attempting to recover the old Boolean checker equation from every new mathematical proof would retain the implementation dependency we seek to remove.

4 Symbolic configurations and the guarded frame rule

4.1 The maintained representation

A symbolic configuration is

$$s = (q, h, L, R),$$

where q is the machine state, h says whether the head is oriented onto the left list, and L, R are lists of symbolic segments. A segment is a literal bit, an affine run of $an + c$ equal bits, or a register-file box. The maintained function $\mathrm{conc}_{n,v}(s)$ interprets the segments as a concrete list-zipper configuration for every natural gap parameter n and register file v.

The maintained symbolic step function sstep_M returns either failure or a new configuration with a Boolean progress flag. That flag is conservative: true guarantees at least one concrete machine step; false does not assert that the concrete step count was zero. In particular, whole-run scans may represent many concrete steps even when their generic lower bound is zero.

Definition 4.1 (Appending an uninspected left frame)**.** For a segment list F, put

$$A_F(q, h, L, R) = (q, h, L +\!\!+ F, R).$$

Call s safe when it is not of the form $(q, \mathsf{true}, [], R)$. This is exactly the executable predicate `safeLeft`.

Theorem 4.2 (Exact guarded frame commutation)**.** *For every transition function M, branch parameter b, safe symbolic configuration s, and frame F,*

$$\mathrm{sstep}_{M,b}(A_F s) = \mathrm{map}((t, m) \mapsto (A_F t, m), \mathrm{sstep}_{M,b}(s)).$$

The equality preserves failure, the complete successful endpoint, and the exact progress Boolean.

Proof. Unfold the maintained symbolic step and split on head orientation and the leading segment. For a literal bit, any read or write concerns that bit, and appending F commutes with the list construction. For a run, either a self-loop transfers the complete run to the opposite side, or the existing peeling operation exposes one cell; both operations commute with appending F. The auxiliary lemmas `peelLeft_addLeft` and `peelRight_addLeft` express this directly.

At the rightmost register box, both the register oracle result and the literal fallback preserve the appended frame. The oracle can prepend a bit to the explicit left fragment, but cannot inspect

its appended tail. A box in an unsupported position remains unsupported on both sides. Failure therefore commutes as well as success.

The sole excluded branch is an empty left list with the head oriented left. The unframed executor inserts a blank in that branch, whereas the framed executor would read the first segment of F. The safety premise rules out exactly this branch. The Lean proof checks these cases against the maintained `KRuntimeCore.sstep`, without modifying its semantics. □

Proposition 4.3 (The guard cannot be deleted)**.** *For every M, b, choose $s = (0, \mathsf{true}, [], [])$ and $F = [\mathrm{bit}(1)]$. The equality in Theorem 4.2 is false.*

Proof. The unframed step reorients across an invented zero and retains the appended one on the left after mapping. The framed step reorients across the actual one, leaving an empty left list and a one on the right. The symbolic configurations are distinct, independently of the transition table. This is a checked theorem in both proof components. □

4.2 Why left normalization is forbidden here

The existing left normalizer may discard trailing blank cells because an isolated tape has blank exterior. That operation is not compatible with later appending a nonblank frame. For example, removing the sole zero of $L = [0]$ before appending $F = [1]$ changes $[0, 1]$ into $[1]$. Therefore the new local proof never applies unframed `normL` to an open left fragment. Only the right endpoint is normalized, using the maintained exact equality `flatT_norm`.

5 Finite replay and strictly positive execution

Define `runLocal` by a fixed number of guarded symbolic steps. Zero steps returns the same configuration with progress false. A successor step first checks `safeLeft`; it then applies the maintained symbolic step and recursively executes the remainder, combining progress flags with Boolean disjunction.

It is intentional that the final configuration may have an empty explicit left list. The guard is required before each executed symbolic step, not at the final cut. This allows the local trace to stop exactly before it would inspect the caller's frame.

Theorem 5.1 (Sound local replay)**.** *Assume the existing register-box contracts for M. If*

$$\mathrm{runLocal}_{M,b}(k, s) = \mathrm{some}(t, m),$$

then for every frame F, gap n, and register file v valid for branch b,

$$\mathrm{Reach}_M(\mathrm{conc}_{n,v}(A_F s), \mathrm{conc}_{n,v}(A_F t)).$$

If $m = \mathsf{true}$, the relation is Reach^+_M.

Proof. Induct on k. The zero case is reflexivity. In the successor case, Theorem 4.2 transports the symbolic step. The maintained `sstep_sound` gives its concrete `StepOk` contract. Compose that reachability with the induction hypothesis. If the accumulated Boolean is true, at least one of the first-step flag and the tail flag is true. Use the corresponding positive composition lemma. These are precisely the two branches in `runLocal_sound`. □

All twenty delivered local traces happen to have progress true. The Home composition below gives a second independent reason for positive progress of the complete macro. Positivity is retained explicitly because it is needed when a target halting bound is used to reflect source halting: allowing a source loop to be simulated indefinitely by zero target steps would invalidate that argument.

6 The improvement: a flexible return-state contract

6.1 Read-only return words

The maintained `KHome.LeftWord M q W q'` relation says that the machine scans the finite word W leftward, preserves every scanned symbol, and changes state from q to q'. We add a simple executable checker `wordEnd`. It follows a row only if the row writes the same bit and moves left; its returned state is the state after the word has been consumed.

Lemma 6.1 (Return-word soundness)**.** *If* $\mathrm{wordEnd}(M, q, W) = \mathrm{some}(q')$*, then*

$$\mathrm{LeftWord}(M, q, W, q').$$

Proof. Induct on W. The empty word has the identity relation. For a nonempty word, checker success gives the exact unchanged-symbol left-moving transition and a successful check of the tail. These are the two premises of the maintained `LeftWord.cons` constructor. □

6.2 Generalized Home completion

Let $\mathrm{Boundary}(M)$ be the already proved standard Home boundary contract from state 2 across the left marker and blank exterior to dispatch state 84. The following result has no fixed initial return state.

Theorem 6.2 (Return through any certified read-only word)**.** *Suppose* $\mathrm{Boundary}(M)$ *and*

$$\mathrm{LeftWord}(M, q, \mathrm{rev}(P) +\!\!+ [1], 2).$$

Then for every right tape R and every number z of exterior blanks,

$$\mathrm{Reach}^{+}_{M}\left(\mathrm{zl}\big(q, (\mathrm{rev}(P) +\!\!+ [1]) +\!\!+ 0^{z}, R\big), \mathrm{zc}(84, [1], P +\!\!+ R)\right).$$

Proof. The maintained blank-padding equality removes 0^z. Apply `LeftWord.sound` to scan $\mathrm{rev}(P)+\!\!+$ $[1]$ without changing any bit. Its endpoint is the state-2 boundary configuration with right tape $1 :: (P +\!\!+ R)$. Compose with the positive boundary theorem. Reversal restores P in its original orientation. □

The far-left exterior remains blank. The theorem does not assert correctness for an arbitrary nonblank exterior beyond the marker; the existing Home boundary proof uses that blank boundary. The unchanged portion *inside* the frame may be any finite P satisfying the return-word condition.

The old proposal forced $q = 2$. The literal machine also has the rows

$$M(1, 0) = (0, L, 1), \qquad M(1, 1) = (1, L, 2).$$

State 1 therefore skips leading zeros in the leftward reading of the prefix, then enters the usual Home scan at the first one. The marker itself guarantees a final one even when the prefix has no one. This permits several local bodies to stop sooner, before consuming an otherwise irrelevant part of the prefix.

6.3 A three-step example serving 181 sites

Consider the template at state 42 for `continue 2`, with explicit suffix $S = 11$. Initially the local left fragment is [1], and the right tape begins with the final 1 of the suffix followed by the gap. The three symbolic operations are:

1. At state 42, use $M(42, 1) = (0, L, 16)$: erase the current one and move left into state 16.
2. At state 16, reorient from the explicit left one onto the right list. This is a representation step and leaves the machine state unchanged.
3. Use $M(16, 1) = (0, L, 1)$: erase that one and stop with an empty explicit left fragment in state 1.

The resulting right prefix is 00. The complete finite trace, rather than this informal description, is reduced against the literal table in `template03`. The suffix contributes two additional gap cells; the preserved prefix is scanned by the return-word theorem. This one template is attached to 181 current source occurrences.

The return scan begins after this local cut and uses only unchanged-symbol transitions. The two erasures belong to states 42 and 16; state 1 preserves the caller's prefix. The representation step is counted among the three symbolic operations but makes no physical machine move.

7 Source-path framing and exact gap arithmetic

The machine proof must implement the source action on the entire program counter $P +\!\!+ S$. Checking only that the local trace reaches a convenient word is insufficient: a break could carry into P, or stripping could remove a bit of P.

Write $\#_b(S)$ for the number of occurrences of bit b in S. The source uses reverse-list stripping and binary increment. The package proves the following facts directly against the maintained `RegisterPath` definitions.

Theorem 7.1 (Stripping and increment preserve a sufficient prefix frame)**.** *For every prefix P and suffix S:*

$$k \le \#_b(S) \Longrightarrow \mathrm{stripPath}(b, k, P +\!\!+ S) = P +\!\!+ \mathrm{stripPath}(b, k, S),$$
$$0 \in S \Longrightarrow \mathrm{incPath}(P +\!\!+ S) = P +\!\!+ \mathrm{incPath}(S).$$

Proof. For stripping, prove the reverse-list append identity by induction on the explicit suffix and the remaining stripping count. The count premise ensures that the target count becomes zero before the recursion can enter the appended prefix. Reverse the equality to obtain the path statement.

For increment, induct on the least-significant-first suffix. A zero ends the carry immediately. A one changes to zero and invokes the induction hypothesis. The presence of a zero in the suffix guarantees that the carry cannot escape into the appended prefix. Reverse again. □

Consequently, a continue template uses

$$k \le \#_1(S), \qquad E = \mathrm{continuePath}(k, S),$$

while a break template uses

$$k \le \#_0(S), \qquad 0 \in \mathrm{stripPath}(0, k, S), \qquad E = \mathrm{breakPath}(k, S).$$

Under these premises the complete source result is exactly $P +\!\!+ E$. All twenty templates also check $|E| \le |S|$.

Two tiny formal counterexamples explain the guards. With $P = 1$, $S = 0$, continuing once must strip a one from the prefix. With $P = 0$, $S = 10$, breaking once strips the final zero and then carries out of the remaining all-one suffix. Both unguarded prefix-preservation claims are false, and both false claims are rejected by the delivered Lean counterexamples.

7.1 Uniform gap parameter

The local templates use a gap $n + 16$. The maintained whole-site representation uses

$$n_0 + (64 - |Q|), \qquad Q = P +\!\!+ S, \quad |Q| \le 48.$$

Instantiate the local parameter with

$$n = n_0 + 48 - |Q|.$$

The length bound makes the subtraction safe and gives

$$n + 16 = n_0 + (64 - |Q|).$$

After the action, the local right endpoint has gap

$$n + 16 + |S| - |E| = n_0 + (64 - |P +\!\!+ E|).$$

These equalities are proved with natural-number subtraction, not silently treated as identities over integers. The file `KPathFrame.lean` supplies the corresponding rebase lemmas, and the standard caller theorem performs both input and output rewrites.

8 The complete twenty-template library

The following table lists the complete library. q is the routed formal entry state, S the explicit suffix, E the source result suffix, r the local return state, and j the number of symbolic operations. Uses are source occurrences. The empty word is ϵ.

ID	q	S	Operation	E	r	j	Uses
0	62	111	CC3	ϵ	1	8	257
1	13	11	CC2	ϵ	1	6	1
2	108	001	CB1	1	1	5	86
3	42	11	CC2	ϵ	1	3	181
4	83	111	CC3	ϵ	1	5	86
5	138	111	CC2	1	2	11	4
6	141	11	CC2	ϵ	1	3	29
7	57	11	CC2	ϵ	1	3	267
8	80	0111	CC3	0	1	19	174
9	72	11	CC2	ϵ	1	6	175
10	126	11	CC2	ϵ	1	3	29
11	47	11	CC2	ϵ	1	3	29
12	139	11	CC2	ϵ	1	3	81
13	108	101	CC2	ϵ	1	5	8
14	103	0011	CB1	1	2	24	4
15	122	1	CC1	ϵ	1	1	6
16	103	0110111	CB1	100	2	39	2
17	103	00111	CB1	1	2	29	2
18	96	0011111	CB1	1	2	39	1
19	162	0011	CB1	1	2	24	1
				Totals		239	1,423

Fourteen templates return at state 1 and six at state 2. Their occurrence counts are 1,409 and 14. The maximum local trace length is 39. A finite additional search admitting the other existing Home scanner states $3, 5, 9, 15, 23, 34, 50$ finds the same library. This supports the chosen simple interface; it does not prove minimality among all possible proof decompositions.

The newly included occurrence is `1000111/CC3`. The only literal control fallbacks are `001/CB1` and `11011/CB1`. Their source indices in the accepted site list are 1 and 4724. The fallback list is a semantic boundary of this selected library, not a claim that those cases cannot be generalized by a different invariant.

9 Exact caller attachment and whole-tape contracts

The finite template data alone do not prove that a particular source leaf may use a template. The Lean caller checker therefore keeps the following information and checks it against current definitions:

1. The complete source word Q, its chosen prefix P, and the template index, with $Q = P +\!\!+ S$ and $|Q| \leq 48$.

2. The exact result of `RegisterPath.walk` on the maintained `p215`, including the operation, argument, and consumed depth $|Q|$.

3. The actual primitive-entry state obtained by following the full prefix of Q through the maintained role array. Premature primitive edges and missing roles fail the selector.

4. The unchanged-symbol return word from the template's return state over $\mathrm{rev}(P) +\!\!+ [1]$ to state 2.

5. The template's source guard, canonical entry shape, and exact normalized right output. These last facts are proved once for each of the twenty template metadata records.

The role selector follows the existing Source193 method [8]; it does not identify callers by a short suffix. Its agreement with `RolePath` is a generic induction. The independent audit additionally rebinds every complete primitive route, including all non-template sites, against the 234-role source decoder and all 430 formal actions.

Theorem 9.1 (Certified source caller)**.** *For every caller in the delivered checked list, every natural gap parameter, and every valid register file, its selected template has the exact source continuation and a strictly positive execution of the concrete maintained machine from the full primitive-entry tape to the full successor dispatch tape.*

Proof. Kernel reduction of the caller Boolean yields the exact split, source walk, role state, and return word. The template Boolean yields the successful local replay. Apply guarded frame transport and Theorem 6.2. The concrete `KTape.boxLemmas` instance and the maintained Home readiness checker discharge the engine and boundary premises. Right normalization is converted to equality of complete right tapes using `flatT_norm`. The source framing lemmas give $P +\!\!+ E$, and the gap rebase identities put both endpoints in the maintained standard form. □

The top-level Lean interfaces are `all_caller_contracts`, `all_caller_standard`, and `caller_bindings`. The first two quantify over unbounded natural register values and gaps; the third records the exact source-walk and route equalities. The additional `all_caller_routes` theorem produces witnesses of the maintained `RolePath` relation. The 1,423 source occurrences are finite because the program is finite; the execution semantics is not restricted to sampled values of those unbounded parameters.

The checked result applies to each member of the delivered caller list. The independent audit also checks the list's exact attachment to the distinct source occurrences. A combined kernel theorem covering every outcome of the whole source program remains an integration obligation; the present component does not replace it merely with a list-length check.

The independent Python checker has a different role. It supplies readable extraction, source expansion, exact label binding, rediscovery, and deliberate mutation checks. It is not part of the logical trust base of the Lean theorem. A generator error cannot turn a failed kernel equation into a successful proof.

10 The Rocq-compatible component

The second component proves the same symbolic frame principle against definition slices extracted verbatim from the maintained Rocq runtime. It proves exact option-map commutation, guarded local replay, progress preservation, a generic semantic run interface, and the necessary-guard counterexample. It also checks all twenty local trace equations and their universally framed forms.

The sparse Rocq source retains a historical `Table230` filename. That name is not used as evidence of machine identity. The extraction audit accounts for the fifteen disabled sparse states and compares every working action with both the current Lean table and the accepted native K215 table. Lean's literal list has 226 positions because the final disabled positions are omitted; the actual set of 430 working actions agrees.

This component was freshly checked with **Coq 8.18.0, OCaml 4.14.1**. Five generic theorem reports and twenty framed-template reports are closed under the global context. Recursive `coqchk` reports no axioms. The delivered production-import adapter has the same shared proof body, but it has not been compiled in the production Rocq 9.1 environment. Nor has the concrete-tape semantic interface in this extracted component been instantiated with the production reachability relations. Those are explicit remaining adoption tasks, not hidden assumptions of a claimed endpoint.

The asymmetry is deliberate and documented: the Lean component reaches concrete whole-tape caller semantics; the Coq compatibility component currently verifies exact symbolic transport and the finite template library. Sharing the finite data avoids maintaining two unrelated trace descriptions, while each assistant checks its own definitions and proof terms.

11 Fresh validation and trust boundaries

The supplied evidence separates three validation layers:

1. **Lean 4.32.0 component.** The complete control-contract import closure contains 51 local source modules: 20 byte-identical upstream dependencies and 31 new modules, including 23 finite caller batches. The checked interfaces cover the generic frame argument, twenty concrete templates, source framing, exact whole-tape caller contracts, and maintained role-route witnesses. The compiler commit is `8c9756b28d64dab099da31a4c09229a9e6a2ef35`. The final build record binds each source, imported object, generated object, and raw compiler log. This is a component closure, not a reduced endpoint closure.

2. **Coq 8.18.0 compatibility component.** Five generic theorem reports and twenty framed-template reports are closed under the global context. The recursive checker `coqchk`, run with `-silent` and `-o`, reports no axioms for `K215FrameTemplates`. The literal table and extracted definition blocks are separately bound to the maintained sources in `EXTRACTION.json`.

3. **Independent finite audit.** Re-expansion reproduces all 4,751 source leaves, every full primitive route, and all 430 working machine actions. The earlier 35-template library and the new 20-template library are both rediscovered from the eight packaged input snapshots. The emitted data agree byte for byte with the delivered records.

The first monolithic 1,423-caller kernel decision was terminated by the environment's 8 GiB memory limit (exit 137). It supplies no accepted proof artifact. The final design checks 23 batches of at most 64 callers and combines their results by `List.all_append`; the final batch has 15 entries. This change preserves the statements and the kernel-checking boundary. The failed attempt is retained as a clearly marked development record so its status cannot be confused with the successful batched checks.

No current full Lean endpoint, production Rocq 9.1 endpoint, or combined all-outcome semantic-coverage theorem is claimed by these component records.

The Lean finite proofs use kernel-checked decisions. No `native_decide`, compiler-trust axiom, `sorry`, or added axiom is used to certify the new results. The general frame equalities require at most propositional extensionality; the complete concrete caller theorem uses the standard inherited profile

`propext`, `Classical.choice`, and `Quot.sound`. The standard library and proof kernel remain the usual toolchain boundary. Official Lean documentation explains the distinction between kernel reduction and native decision evaluation [10].

The archive distinguishes successful final build records from development work. It includes fresh source, log, and object hashes in the build evidence; generated object files themselves need not be distributed. Reproduction creates a fresh output directory and rebuilds from source. The recorded timings are single-run observations of this component, not controlled benchmarks against the former complete site family.

Six attachment mutations are rejected by the independent finite audit: changing the routed state, local gap offset, endpoint gap delta, result suffix, return state, or complete source word. Separate examples reject the unsafe empty-left frame step, invalid left normalization, insufficient stripping, and a carry into the prefix. These checks target specific known failure modes rather than merely duplicating the implementation on arbitrary sample inputs.

12 Integration plan

12.1 First preserve a small stable interface

Add the generic frame and return-word modules adjacent to the maintained runtime. Their useful public interface is a concrete execution contract, not an executor-fuel equation. Keep the existing symbolic step and tape interpretation unchanged. Retain the source-action append lemmas in a small path module, so runtime framing does not need to import the logical calibration.

The current packet is additive. The named endpoint files and production site batches are not overwritten. The accompanying sources and generators make the intended next change reviewable without removing the existing certified route before the replacement is connected.

12.2 Attach to semantic residual coverage

Use the existing decoder-prefix theorem to reach the exact primitive port. Then use the new standard caller theorem for the 1,423 selected controls, and the existing literal semantic suffix contracts for the two control exceptions, all 1,900 increment occurrences, all 2,850 decrement outcomes, and the halt occurrence.

The source totality proof, decoder routing, consumed-word convention, register permutation, spare-register extension, and literal terminal-row check all remain required. In particular, exhausted decoder fuel must not be treated as a halting leaf. This is the same distinction retained in the existing semantic residual work.

The Source193 residual component already provides an appropriate integration direction, including the full consumed word and separate prefix execution. Its archived component checks do not automatically certify a combination with the new frame library. Select one current semantic simulator interface, adapt the new contracts to it, then rebuild that combined source closure.

12.3 Maintain one generator and one finite format

The generated records should contain data only: source word, prefix, template identifier, source instruction, and the exact local template. The proof assistant checks their interpretation. Emit both Lean and Rocq traces from the same finite description, but do not assume one assistant's proof validates the other's tape semantics. Keep the state-label map explicit and compare all transition actions during regeneration.

The packet supplies the independent rediscovery script, template emitters, caller emitter, source snapshots, and hashes. A maintained adoption should keep the generators synchronized with any chosen production layout. Source text removed from site batches must be counted together with any new generic proofs and metadata; a shorter batch alone is not a net maintenance saving.

12.4 Finish with the actual endpoint clients

After the semantic adapter is installed, rerun the current transitive Lean closure and typed endpoint clients, and the production Rocq 9.1 build and recursive checker. Compare the exact halting and nonhalting theorem statements and their assumptions. Recheck the dense-table identity and blank initialization on the same literal accepted machine. This is where an actual replacement can be claimed.

Because the pinned tree already contains later computed-tree, compiler, and seed changes, the final current-source rebuild should cover those adopted changes too. Reusing the historical phase-fifteen module count as a current build manifest would omit the distinction between adoption and complete verification.

13 What the reduction measures

Replacing 1,423 literal control executions by twenty reusable bodies and retaining the two exceptions gives

$$20 + 2 + 1900 + 2850 + 1 = 4773$$

candidate expensive execution obligations. Relative to 6,176 unshared outcome checks, this is 1,403 fewer, approximately 22.72%. The earlier 35-template decomposition would give 4,789; the additional flexible-return refinement removes sixteen more expensive obligations.

The library itself has 42.86% fewer distinct traces and 46.17% fewer symbolic trace operations than that earlier frame proposal. These are exact counts of different finite objects. They must not be added together or presented as percentages of the entire proof development.

Every original outcome still needs source coverage. Many caller checks become small equations about lists, source walks, routes, and return words. This can improve maintainability and may reduce repeated kernel computation. Actual full build time also depends on lookup representations, import cost, proof reduction strategy, object size, and batching. No monotone relation between source bytes and build time is assumed. The repository's own computed-tree experiment already illustrates that a substantial source-text reduction can make a particular checked batch slightly slower [4].

For an adoption benchmark, compare clean builds at a fixed toolchain and hardware allocation; record total elapsed time, peak resident memory, the slowest site batches, generated source bytes, handwritten source bytes, and object bytes. Report both the one-time generic proof cost and the marginal cost per reused occurrence. The package's component build is evidence of correctness and reproducibility, not that benchmark.

14 Further research questions

1. **Can the return-language interface compress the caller data?** The exact `wordEnd` check already recognizes a regular read-only return language. A small automaton proof could replace repeated prefix scans without weakening the full-word route attachment. The useful

optimization target is total proof cost, including the automaton theorem and occurrence witnesses.

2. **Can the two remaining CB1 controls share a boundary contract?** Their stripping or carry reaches the selected frame boundary, so the present no-carry suffix invariant intentionally excludes them. A separate contract that moves the marker or explicitly rewrites a bounded prefix might absorb them. Any such contract must preserve the exact overflow and gap convention.

3. **Which increment/decrement families admit comparable framing?** These dominate the remaining outcome count. They interact with register marking, spare tails, and the positive-decrement boundary shift. A promising next experiment is to reuse the established register oracle contracts while factoring only the program-counter suffix and return segment. Full source words and physical-register indices must remain attached.

4. **Can proof-library selection be optimized with a certified cost model?** We found twenty templates under a particular empty-left cut and return-language design. No optimality theorem is asserted. A finite candidate catalogue might allow a checked set-cover problem over the 1,425 control occurrences, with costs that include trace size and caller overhead rather than only template count.

5. **Can the two proof assistants share more certificate data without sharing trust?** A compact, precisely specified trace-and-attachment format could feed separate Lean and Rocq checkers. The checkers should agree on label mapping, disabled states, branch assumptions, and terminal conventions. Their semantic soundness proofs would remain independent.

6. **How much of this runtime argument can be internalized arithmetically?** The concrete finite data are elementary; the generic simulation and ZFC calibration have different proof requirements. A separate project could make the claimed object-theory equivalence explicit in a chosen weak arithmetic or in encoded ZFC proof syntax. That task should retain its own statement and evidence boundary rather than be inferred from this refactoring.

15 Artifact guide and reproduction

The archive root contains this LaTeX source and its compiled PDF under `article/`; new Lean sources and exact upstream prerequisites under `lean/`; the Coq compatibility component under `rocq/`; independent extraction and census tools under `audit/`; machine-readable records and raw logs under `evidence/`; and retained attribution under `licenses/`.

The root `README.md` specifies the exact build commands. `verify.py` verifies the delivered manifest and reruns finite extraction into a separate work directory. With an explicitly supplied Lean 4.32.0 compiler it rebuilds the Lean component; the Rocq-compatible component has its own checker command accepting `coqc` and `coqchk`. The source census is reproduced against the pinned repository, not against the intentionally smaller component snapshot.

The most useful integration entry points are:

Source/interface	Purpose
`LeftFrame.lean`	Guarded step equality, run transport, return-word checker, Home composition
`FramedTemplates.lean`	Twenty literal local traces checked against `tm215`
`K215FrameEngine.lean`	Concrete register-engine and Home premises
`KPathFrame.lean`	Source-action prefix preservation and gap arithmetic
`FrameContracts.lean`	Whole-tape semantics and exact source/role caller interface
`FrameCallerCore.lean`	Twenty metadata records and the exact caller checker
`FrameCallers.lean`	Composition of 23 checked caller batches and 1,423 contracts
`FrameRouteSound.lean`	Checked caller routes yield maintained `RolePath` witnesses
`K215FrameTemplates.v`	Same twenty symbolic traces and universally framed equations in Coq

The new module boundaries preserve the central advantage of the existing construction: the sophisticated logical calibration remains independent of local tape representation. The proposed simplification removes repeated runtime reasoning at the point where a small, exact contract can replace it.

References

[1] ProveIt, *Formal correctness of the exact 215-state machine*, pinned verification README. https://github.com/VladimirReshetnikov/ProveIt/blob/cbf94cb552a3b49509e0eee80e7455d4eb5974c6/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/README.md.

[2] ProveIt, *Fifteenth K215 proof-simplification checkpoint*. Same pinned repository, `Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/phase15/README.md`.

[3] ProveIt, *Round 13 runtime: factor at the first empty-left Home boundary*. Same pinned repository, `Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/proposals/round13-left-frame/runtime/README.md`.

[4] ProveIt commit `775267c38acd60f24d24c9512de54e4bc52f2d0d`, *Lean: decoder trees computed from their programs, not written out*. https://github.com/VladimirReshetnikov/ProveIt/commit/775267c38acd60f24d24c9512de54e4bc52f2d0d.

[5] ProveIt commit `fa92b88f8e622a22711f68d4f22f281cd0d18c40`, *Lean: the 215 and 217 enumerators compile C222 directly*. https://github.com/VladimirReshetnikov/ProveIt/commit/fa92b88f8e622a22711f68d4f22f281cd0d18c40.

[6] ProveIt commit `ff928ef0ecce89f961cd4ab8a049b07083c5bace`, *K215 Lean: the full seed directly on the original model*. https://github.com/VladimirReshetnikov/ProveIt/commit/ff928ef0ecce89f961cd4ab8a049b07083c5bace.

[7] ProveIt, *K215 proof simplification: semantic interfaces and smaller imports*. Same pinned repository, `Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/proposals/semantic-proof/README.md`.

[8] ProveIt, *Batch 215: exact residual contracts and shared scheduler proofs*, reconciliation of Sources 193–194. Same pinned repository, `SetTheory/Cardinals/docs/reports/set-theory/zfc-closure-kernels/intake-215.md`.

[9] Maintained proof sources `KRuntimeCore`, `KHome`, `KHomeReady`, `KZip`, `KTape`, `RegisterPath`, `K215Decoder`, and `DecoderPaths`, under `Computability/BusyBeaver/Lean/BusyBeaver/ZFC/` at the pinned commit. Exact component source hashes are included in the archive.

[10] Lean Language Reference, *Validating a Lean Proof*. https://lean-lang.org/doc/reference/latest/ValidatingProofs/. Consult the pinned compiler's corresponding documentation when reproducing a version-specific check.